

FRHIDS: Federated Learning Recommender Hybrid Intrusion Detection System Model in Software-Defined Networking for Consumer Devices

Himanshi Babbar^{ID} and Shalli Rani^{ID}, *Senior Member, IEEE*

Abstract—In the past few years, numerous methods of attack against recommendation systems have been developed. Cellphones, smart devices, and self-driving cars are instances of distributed IoT consumer devices that generate massive amounts of data on a daily basis and pose security threats to the cloud server. Due to the higher exchange of data, the challenges in this domain lead to increased security issues. Therefore, intrusion detection systems are important for the security and privacy of IoT consumer devices and hence to the cloud server. Due to the prediction, classification of attacks and recommendation of malware devices, the accuracy of machine learning and deep learning approaches for research in security for IoT consumer devices has gained tremendous popularity. Federated learning (FL), is a privacy-preserving decentralized learning technique that does not transport data but instead trains the model locally before sending the parameters to a cloud server, which helps in ensuring the security of data. However, communication channels can still be attacked by hackers, so blocking malicious data is a major requirement for the cloud server. In this paper, a federated learning recommender hybrid intrusion detection system (FRHIDS) model has been proposed that detects the attacks on the SDN network incoming from the IoT consumer devices and recommends that the safety devices transmit the decrypted data to the federated cloud server. In this model, the preservation of the security and privacy model parameters by utilizing the process of testing and training has been implemented. Simulation shows that the proposed approach's well-designed recommender system has outperformed state-of-the-art models. The performance of the proposed technique is evaluated based on its computational complexity and validation, which have shown 12% improvement over the already existing techniques.

Index Terms—Recommender system, federated learning, intrusion detection system, hybrid deep learning model, consumer devices.

I. INTRODUCTION

THE RECOMMENDER System has nowadays become the essential channel for people to extract information that can directly influence the perceptions of people while recommending consumer devices. Therefore, there are numerous vulnerabilities in the cloud server that can be exploited to

modify the recommendation of consumer devices to transmit the data. Due to the remote connectivity of the consumer devices to the cloud server, hackers try to target the consumer device and hence the server. It is a major requirement for the cloud server to recognize malicious and safe data from the devices [1], [2], [3]. The known attacks, such as the Mirai botnet and its more ongoing variants show the necessity of enhancing IoT device security to safeguard massive IoT-enabled systems [4]. In recent decades of IoT, the employment of machine learning methods has indeed been discussed extensively for the recognition and mitigation of these attacks due to the growth of such increasingly sophisticated attacks. In fact, using a variety of techniques (such as neural networks) to infer probable attacks [5], the implementation of ML techniques has been suggested in ongoing studies to enhance the recognition capabilities of well-known network intrusion detection systems (IDS). A NIDS is software installed at the network gateways that are designed to determine whether the traffic is considered to be an attack or not and to protect federated cloud server information systems. “Signature-based” and “anomaly-based systems” are two significant approaches that can be integrated with NIDS [6], [7], [8].

Depending on certain characteristics in the Internet traffic, including the number of bytes or the number of 1s or 0s, signature-based IDS can identify attacks. Additionally, it recognizes attacks depending on the malware's identified harmful instruction sequence. The anomaly-based IDS was developed to identify unidentified malware attacks, whereas the IDS detects characteristics that are known as signatures [9]. The fundamental conceptual difference between signature-based and anomaly-based detection is that the former is designed to find a specific, known attack, while the latter is not ideal for detecting new, unknown attacks [10], [11], [12]. For example, using the multi-classification model to tag traffic data, implies that the assault has already been understood. It will be unable to detect any new malicious events, resulting in data leakage [13]. The latter system, on either side, has a greater capability for identifying unknown attacks and performs better in intrusion detection analysis [14].

The focus of this article is on data poisoning attacks for anomaly-based detection methods and hence recommends only the safety devices to transmit the data whereas to block the malicious nodes' data [15].

Manuscript received 1 June 2023; revised 4 October 2023; accepted 29 October 2023. Date of publication 1 November 2023; date of current version 26 April 2024. (Corresponding author: Shalli Rani.)

The authors are with the Chitkara University Institute of Engineering and Technology, Chitkara University, Rajpura 140401, India (e-mail: himanshi.babbar@chitkara.edu.in; shalir79@gmail.com).

Digital Object Identifier 10.1109/TCE.2023.3329151

A. Main Contributions

The main contributions of the paper:

- 1) In this paper, a FRHIDS model is proposed to detect malicious attacks on consumer devices on the cloud server. Later, a federated cloud server approach is developed which generates the detailed intrusion detection model by overtaking the benefits of the benign data resources from the SDN network. Malicious nodes' are blocked from sending the data to the cloud through the recommended approach for a cloud server.
- 2) To improve the learning performance of the proposed model, the computational complexity is calculated based on the overall effectiveness of the proposed FRHIDS model with regard to the baseline models. The regularization technique is applied to the proposed model to reduce the complication of overfitting.
- 3) The experimental analysis and results are conducted to compute the efficiency of the proposed model in comparison to the other solutions for pervasive IoT consumer devices and hence recommendations to the cloud are made. The results have shown an improvement of 12% in the proposed model with regard to accuracy, precision, recall and f1-measure.

B. Paper Organization

The rest of the paper is organized as: Section II brief about the preliminaries of the neural collaborative recommended filtering model and work carried out by the existing authors; Section III discusses the methodology of the proposed work; Section IV explains the federated recommender hybrid IDS framework; Section V showcase the result analysis and performance evaluation; Section VI depicts the comparative analysis of proposed with existing literature; Section VII concludes the paper.

II. PRELIMINARIES

In this, the detailed description of state-of-the-art recommender models is studied based on the intrusion detection recommender model in the SDN network as well as federated recommender learning-based IDS for consumer devices.

A. Neural Collaborative Recommender Filtering Model

In neural collaborative model, authors have used NCRF as primary recommendation model without any loss of generality. Let us assume, N and M are the numbers of domains and consumer devices respectively in the recommender system. Let's say, U and I represent the set of domains and set of consumer devices. Every domain $u \in U$ acquires the embedding vector e_u denotes the latent features. In the same way, each consumer device $i \in I$ acquires the embedding vector f_i . The utilization of Z_{ui} denotes the score predicted between the domain u and consumer device i, which signifies the domain u preference for each consumer device i. In NCRF, Z_{ui} is shown as following in eq. (1):

$$Z_{ui} = \mu(e_u, f_i) \quad (1)$$

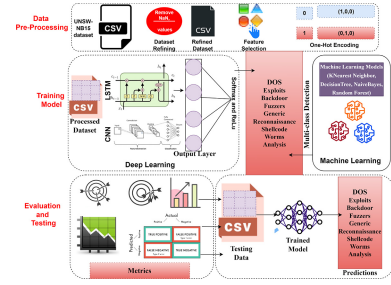


Fig. 1. Proposed Methodology.

where μ denotes the domain and consumer devices interaction function. For modeling the feedback signals with a maximum level of nonlinearity, NCRF utilizes the multi-layer perceptron (MLP) for learning the function μ has been shown in eq. (2):

$$\mu(e_u, f_i) = a_{out}(h^T \phi(e_u \oplus f_i)) \quad (2)$$

in which out , h , ϕ and $\oplus$ indicate the activation functions and the weight vector, i.e., MLP function and concatenation of vector respectively. The use of the sigmoid function as out to restrict the model's output to be in the binary form, i.e., 0 and 1. Furthermore, let us say a_j , W_j signifies the activation function, the weight matrix and the bias vector in the j^{th} of the perceptron respectively. The NCRF with H hidden layers, the MLP function ϕ is as shown in eq. (3):

$$\phi(x) = \Phi_L(\dots(\Phi_2(\Phi_1(x)))\dots) \quad (3)$$

where $\phi_j(x) = a_j(W_j^T x b_j)$. For acquiring the effective performance of recommendation, the utilization of Rectifier (ReLU) as the activation function a_j in all the layers of the perceptron. In the case of FL, there are attacks that are applied to various recommender models as long as the interaction between domain and consumer devices.

III. METHODOLOGY

The idea of a HIDS architecture is proposed initially in this section and the UNSW-NB15 dataset is utilized to train our model.

A. Machine and Deep Learning Intrusion Detection Model

The proposed architecture facilitates the deep learning architecture for the detection of IDS in SDN-IoT. The hybrid deep learning model is deployed for the detection of threats in IoT networks [5] and is tested and trained with higher accuracy and the least false positives. This utilizes the Keras framework with TensorFlow for the implementation in Python. The comparison is made with four ML algorithms and CNN based DL algorithm having one layer of CNN and one layer of LSTM with kernel size '1' for both layers. Rather, we have compared our proposed model with existing literature as shown in Table I.

The proposed model consists of the CNN module, softmax layer, dropout, and hidden layer. The **CNN module** shows the concatenation operation; **Dropout layer** depicts the technique that is used for model prevention from overfitting, **Softmax layer** is the activation function being used with multi-class

TABLE I
CNN-LSTM DESCRIPTION

Algorithm	Layers	Activation Function	Loss	Optimizer	Batch size	Epochs
CNN (Domain1)	59,1	softmax	mse	adadelata	32	500
(Domain2)	49,1					
(Domain3)	52,1					
LSTM (Domain1)	59,1	sigmoid	mse	adam	32	500
(Domain2)	49,1					
(Domain3)	52,1					
Federated Hybrid IDS Model (Domain1)	59,1	softmax	binary cross entropy	adam	32	500
(Domain2)	49,1					
(Domain3)	52,1					

categorization when more than two class labels necessitate classifier.

B. Proposed Federated Hybrid Recommender Model for IDS

Our base recommender model utilizes a hybrid recommended CNN-LSTM deep learning recommendation model, also referred to as a deep learning hybrid model, enabling intrusion categorization of authentic data traffic [16]. The advantages of the hybrid model are it works more efficiently and detects intrusions as compared to other neural network-based models. This model is a combination of two 1-D convolutional layers, one 1-D maximum pooling layer, one 1-D LSTM layer and one fully connected layer. The Rectified Linear Unit (ReLU) function is the activation function employed to train the model in the two convolutional layers. The hybrid architecture consists of 1D CNN and LSTM are depicted.

$$\partial va(t+1) = -\beta g(va) - \beta/m * \partial H/\partial va + n\partial va(t) \quad (4)$$

As shown in eq. (4), va refer to the weights with respect to β and g representing the regularization parameter and learning rate; m and n denotes the total number of samples trained and momentum; t is the updated step. These parameters are updated and tuned by the use of a training process to fulfill the optimal performance. The eq. (5) is shown below:

$$\sigma(y) = \text{maximum}(0, y) \quad (5)$$

The output of the maximum-pooling layer is passed to the 1-D LSTM layer which is then transferred to the 1-D LSTM network. The 1-D LSTM layer receives input from the maximum-pooling layer. The 1-D LSTM network discovers correlations between the extracted characteristics while arbitrarily disabling some weights to avoid overfitting. In order to categorize incursions for identification, the output of the 1-D LSTM layer is sent to the fully connected layer, which

also has a softmax function [17]. This function calculates the probability distribution of m intrusion classes as follows:

$$q_a = \frac{\exp y_a}{\sum_{b=1}^n} \quad \text{for } a = 1, 2, 3, \dots, m \quad (6)$$

where y depicts the input data. The values generated as output of q are between zero and one, with the sum of the outputs equal to one depicted in eq. (6). The number of CNN filters, number of epochs, learning rate, number of LSTM hidden units, drop-connect ratio, batch size and maximum pooling length are the characteristics of the hybrid model that are most crucial. As during the training stage, each of these parameters is learned by doing. The feature extraction phases employ the ReLu functions for both convolutions and pooling. For the first layer we utilized 49*1 for domain1, 59*1 for domain2 and 52*1 for domain3 with 3*3 kernel size and the output size for the LSTM is set to 70. The function softmax is applied to the types of attack. The optimizer “adam” is deployed for gradient descent and dropout value 0.01.

IV. FEDERATED RECOMMENDER HYBRID IDS FRAMEWORK

In order to avoid ambiguity, the basic objective of the proposed FRHIDS scheme is to network numerous domains to collectively build the deep learning IDS model based on the proposed model. The workflow of the attacks against the recommender system in the proposed FRHIDS can be explained into different phases which are described in Algorithm 1:

- 1) **Initialization Process:** In this processing phase, the entire system of the SDN network is accomplished by computing the KeyGenerate(g), the public key $UK = (n, f)$ and private key $PRK = (\gamma, \tau)$, therefore, the secure link between the federated server and SDN network is developed. The federated cloud server chooses an array of parameters m_0 for the deep learning IDS model and various other parameters to train the model. Thus, the learning rate μ , decay rates for moment estimates $v_1, v_2 \in (0, 1)$, loss function ϵ , numerical stabilization Γ and batch size B . Furthermore, the SDN network N_g reports the size L_g of its private data resources H_g to the federated server where $g \in [1, 2, \dots, G]$ where G is the cloud server for training the discriminator I , g indicates SDN generator and therefore, the federated server that evaluates the ratio for the SDN network by $\beta_g = \frac{L_g}{(L_1, L_2, \dots, L_g)}$. Between the federated server and SDN network the integer (N) should be denoted as the total number of communication rounds.
- 2) **Training the local model by SDN controller:** From the federated server, after the model parameters are acquired m^0 and $\mu, v_1, v_2, \epsilon, \Gamma, B$ deep learning model is trained on the SDN controller utilizing their own data resources H_g based on the IDS.
- 3) **SDN network encryption for model parameters:** Every data incoming from the domains B_g is collected in the SDN network and the deep learning model is trained on the SDN network which later will encrypt the parameters m_g^q and deploying the ParaEncrypt($m_{g,x}^q, UK$) where $m_g^q = (m_{g,1}^q, m_{g,2}^q, \dots, m_{g,T}^q)$ and $x \in \tau = 1, 2, \dots, T$.

Algorithm 1 Federated Cloud Server With Data Protection

Input: Protected parameter g , data resources $H_g | g \in G$, communication rounds N .

Output: Deep Learning Model

Initialize: i. the KeyGenerate(g) utilizing the public key $UK = (n, f)$ and private key $PRK = (\gamma, \tau)$

ii. the learning rate μ , decay rates for moment estimates $\nu_1, \nu_2 \in (0, 1)$, loss function ϵ , numerical stabilization Γ and batch size B .

iii. the SDN network N_g reports the size L_g of its private resources of data H_g , to the federated server where $g \in G = 1, 2, \dots, G$

iv. Initialize the communication rounds to $q=1$

For SDN network: for $N \geq q$

1: **procedure** BEGIN:

2: $\forall g \in G$ **do** evaluates the q -th model communication round for the localized model with parameters m_g^q having inputs: $\mu, \nu_1, \nu_2, \epsilon, \Gamma, B$

3: **for** $x \in \tau$ **do**

4: $E_{Pja}(m_{g,1}^q) = \text{ParaEncrypt}(m_{g,x}^q, UK)$ of the deep learning model are trained

5: **end**

6: H_g transmits the model parameters that are encrypted $E_{Pja}(m_{g,1}^q) | x \in \tau$ to the federated server

7: **end**

8: **For Federated Cloud Server:**

9: **for** $x \in \tau$ **do**

10: $ce_x = \text{ParaAggregate}(E_{Pja}(m_{g,1}^q), \dots, m_{g,T}^q, \alpha_1, \dots, \alpha_G)$ the ciphertexts encrypted $ce = ce_x | x \in \tau$ and are transmitted back to the IoT domains

11: **end**

12: **For SDN Network:**

13: **for** $N \geq q \quad \forall g \in G$ **do** $m_g^q = \text{ParaDecrypt}(ce_x, PRK(x \in \tau))$

14: **end**

15: $q = q + 1$

16: Receive error terms E_{em} and E_{gm} within the SDN network

17: **return**

18: Deep learning model having model parameters m^N

Furthermore, the model parameters that are encrypted $E_{Pja}(m_{g,1}^q) | x \in \tau$ of the deep learning model are given the training and then transferred to the federated server by every IoT device; where T symbolizes how many variables are there in the local model?

4) *Federated server aggregation for model parameters:* The ratios and model parameters that are encrypted from all IoT domains, the aggregation of the federated server is done by ParaAggregate ($E_{Pja}(m_{g,1}^q), \dots, m_{g,T}^q, \alpha_1, \dots, \alpha_G$). lastly, the ciphertexts encrypted $ce = ce_x | x \in \tau$ and are transmitted back to the IoT domains.

5) *Model update by SDN network:* Every SDN network acquires the updated model parameters m^q by decrypting the ciphertexts ce and deploying the

ParaDecrypt($ce_x, PRK(x \in \tau)$). Therefore, the parameters are updated by m^q of the local model and transmitted to the federated cloud server.

A detailed deep learning-based intrusion detection model is developed in Algorithm 1 after N rounds (an experimentally defined threshold) of exchanges between both the federated cloud server and SDN network via IoT domains. The private data resources in SDN network H_g are required to compute the encrypted parameters and decrypted tasks which in all needs multiplied operations in every round of communication. In such a way, in the deep learning model, the computational cost is calculated for each SDN network H_g and is linearly proportional to the number of parameters.

A. Algorithm for Hybrid IDS Training on SDN Network

In the training on the HIDS model, the SDN network is depicted as an SDN controller, the LSTM (encoder) O_m having the parameters p_{em} and the CNN (generator) G_m having its parameters p_{gm} . The LSTM is a multi-layer perceptron that maps the flow of traffic f to the latent representation r , The CNN maps the random vector r to the flow generated f . In every global iteration, SDN_m generates the benign and malicious data, if malicious data is encountered error is generated on which it transmits the benign data outputs $O_m(f)$ and $G_m(r)$ post-training the discriminator I and the parameters of its encoder O_m and generator G_m is updated to the federated cloud server. The flow $G_m(r_m), r_m$ is generated from the SDN controller SDN_m and the valid flow $f_m, O_m(f_m)$ is also transmitted.

After the error terms E_{em} and E_{gm} are generated at the SDN network, the controller will update the parameters p_{em} and p_{gm} of the generator G_m and encoder O_m . The parameters that are computed in eq. (7):

$$\delta p_{gm,l} = \frac{-\partial \log(I(G_m(r_m), r_m))}{\partial p_{gm,l}} \cdot \frac{1}{b} \sum_{G_m(r_m) \in G_m(r_m)} \frac{-\partial \log(I(G_m(r_m), r_m))}{\partial G_m(r_m)_n} * \frac{\partial G_m(r_m)_n}{\partial p_{gm,l}} + \frac{1}{b} \sum_{G_m(r_m) \in G_m(r_m)} * p_{m_n} \frac{\partial G_m(r_m)_n}{\partial p_{gm,l}} \quad (7)$$

where $p_{gm,l}$ depicts the l^{th} parameter of p_{gm} . The parameter updated of encoder O_m can be computed in eq. (8):

$$\delta p_{om,l} = \frac{-\partial \log(I(f_m, O_m(f_m)))}{\partial p_{om,l}} \cdot \frac{1}{b} \sum_{O_m(f_m) \in O_m(f_m)} \frac{-\partial \log(f_m, I(O_m(f_m)))}{\partial O_m(f_m)_n} * \frac{\partial O_m(f_m)_n}{\partial p_{om,l}} \quad (8)$$

We have utilized the “adam” optimizer for updating the parameters.

Once the discriminator is trained, the federated server transmits a version of the I to the SDN controller for intrusion identification, as described in the Algorithm 2. For computing the traffic anomaly incoming from the IoT domains, function $D(f)$ is described as the integration of “reconstruction loss” R_S and a loss of discriminator R_I , is depicted in eq. (9):

Algorithm 2 Hybrid IDS Model for Training SDN Network

Input: SDN Network (T,b,s)
Initialize: p_{gm}, p_{em} for G_m and O_m

```

1: procedure BEGIN:
2:    $L_m \leftarrow ReadDataset(m)$ 
3:   for each  $n \in [1, T]$  do
4:     for each  $s \in [1, \frac{L_m}{b}]$  do  $f_m \leftarrow FlowSample(L_m, b)$ 
5:     set  $r_m$  as a randomized vector
6:      $O_m(f_m) \leftarrow Encoder(f_m)$ 
7:      $G_m(r_m) \leftarrow Generator(r_m)$ 
8:     Receive error terms  $E_{em}$  and  $E_{gm}$  within the SDN
       network
9:     Transmit  $(f_m, O_m(f_m))$  to the federated cloud server
10:     $p_{gm} \leftarrow GeneratorUpdate(E_{gm}, G_m)$ 
11:     $p_{em} \leftarrow EncoderUpdate(E_{om}, O_m)$ 
12:  endfor
13: endfor

```

TABLE II
SIMULATION ENVIRONMENT

Simulation Parameters	Value
Simulator	Jupyter-Notebook
Programming Language	Python
Number of IoT Domains	3
Number of IoT consumer devices	90
Number of Epochs	500
Data Size	512 Byte
Speed of IoT consumer devices	20 m/s

$$D(f) = \beta R_S(f) + (1 - \beta) R_I(f) \quad (9)$$

where β is the coefficient of weights, $R_S(f) = ||f - G(f)||1$ and $R_I(f) = \sigma(I(f, O(f)), 1) * R_I(f)$ deploys the binary cross entropy loss σ from the discriminator of f considered as a true sample, which later gathers the confidence of discriminator's that a flow is inherited from the true data bifurcation. For every flow that is incoming f , the SDN controller SDN_m evaluates its anomaly score $D(f)$. If $D(f)$ is higher than the preset threshold value which justifies the flow f as abnormal.

V. RESULT ANALYSIS AND DISCUSSION

The final layer in a CNN's architecture works similarly to other neural network activation functions. For such multiclass classification issues, our model employs a softmax function. To obtain the most accurate value for a class, the activation function is used to compute the chances of every class and chooses the greatest value among some of the range of possibilities. During the tuning process, the several parameters are changed. The number of epochs were tested between 100 and 500, and the 500 epochs were selected because it performed better across all data categories (TCP, UDP, ICMP). For three distinct domains, the batch size is 1, and there are 512 nodes within every layer. The metrics are preset to "linear regression equality", the loss is preset to "mse" and the optimizer is preset to "adam" with default parameters. The simulation's inputs are displayed in Table II.

TABLE III
HYBRID MODEL DESCRIPTION

CLASS	Label	TRAINING	TESTING
Normal	0	56,000	37,000
Analysis	1	2,000	677
Backdoor	1	1,746	583
DoS	1	12,264	4,089
Exploits	1	33,393	11,132
Fuzzers	1	18,184	6,062
Generic	1	40,000	18,871
Reconnaissance	1	10,491	3,496
Shellcode	1	1,133	378
Worms	1	130	44
Total Record		175,341	82,332

A. Dataset and Preprocessing

The UNSW-NB15 dataset is deployed to validate the efficiency of the proposed DL model. The dataset wraps the recent types of attacks: Backdoor, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, Worms, and Analysis. Furthermore, the normal traffic in the dataset enfolds the service-based applications including TCP, FTP, UDP, SNMP, etc. and the analysis of traffic is explained for the flows that are cumulative at the time of simulations while developing the dataset. The statistical analysis of the dataset represents the simulation period which shows the number of bytes used for transmission and receiving, the number of packets transmitted and received, type of protocol, source and destination IP addresses that is different for all the flows of data.

B. Dataset Features

This section explains a few features that are showcased which can be gathered in the SDN. In SDN network, from the devices of IoT, the controller can gather only the statistical features. The entire framework is facilitated in producing the final output of the dataset from pcap files to CSV files. The first category, the OVS group, consists of intrusions into the internal SDN network that originate from the outside world. The assaults on the Metasploitable 2 server are included in the second category. The final group is the traffic that is not harmful. There are 1,75,341 and 82,332 records in total for training and testing the entire dataset, correspondingly. The three different domains in Fig. 2 shows the benign and malicious attacks happened in each domain. The domains comprised of various traffic data whose main task is to transmit the data to the SDN network. Therefore, by utilizing the tcpdump tool, pcap files are produced. The features of UNSW-NB15 dataset are extracted by deploying the Argus and Bro-IDS tool. The major categories of the records are benign and malicious. The malicious records are comprised of nine malware families as per the nature of the attack. The number of dataset records is shown in Table III.

The UNSW-NB15 dataset's numeric features are denoted as integers has the value **41**, while the non-numeric characteristics are denoted as strings has the value **3**. We could only train our deep learning models with numerical input values, so the conversion of non-numeric data to numeric features is a must. Non-numeric characteristics are treated similarly in other KDD cup99 and NSL-KDD. We also changed the

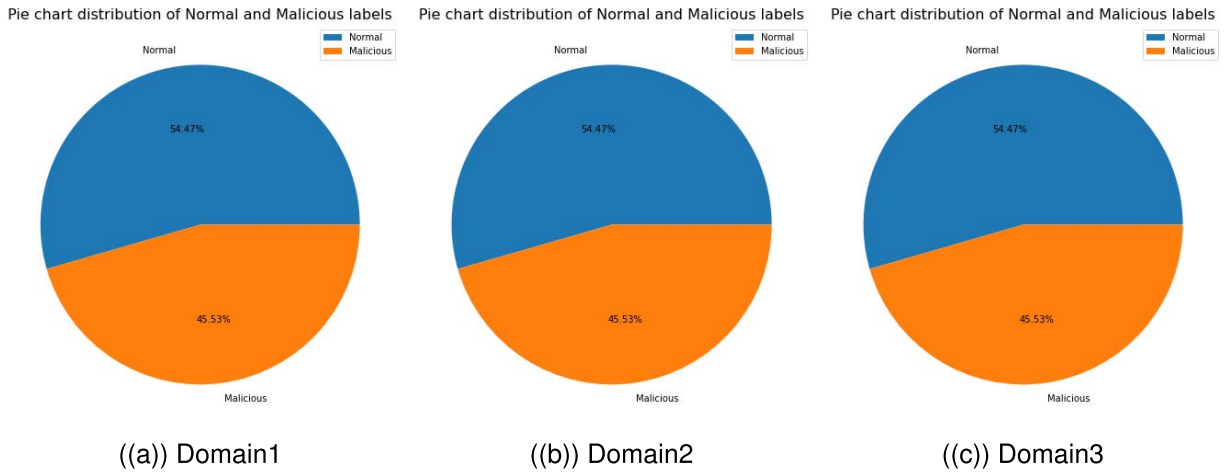


Fig. 2. Normal and Malicious Attacks in three domains.

TABLE IV
EXTRACTION OF 30 FEATURES

F.Number	Name of Attribute	F.Number	Name of Attribute
1	id	16	dinpkt
2	dur	17	sjit
3	proto	18	djit
4	service	19	swin
5	spkts	20	stepb
6	dpkts	21	dtepb
7	sbytes	22	dwin
8	dbytes	23	dcprrt
9	sttl	24	synack
10	dttl	25	ackdat
11	sload	26	c-state-ttl
12	dload	27	is-ftp-login
13	sloss	28	ct-ftp-cmd
14	dloss	29	attack-cat
15	sinpkt	30	label

array's data type to float and eliminated features like 'id' and 'string label'. There are two tools used for the implementation of the dataset. It is comprised of 49 features and 2,540,044 instances which are deployed in four CSV files. The dataset consists of Transmission Control Protocol, User Datagram Protocol, number of source packets, total of source and destination bytes, and Normal and Malicious network traffic. For computing the DL and ML IDS, the dataset is converted into CSV format comprising of benign and malicious traffic in different domains.

The Table IV shows the features that are processed from the raw network packets (pcap files) and produces the attributes or features of the flow packets in the network. In this experiment, out of 49 only 30 features are elected from the dataset. These features are elected based on the data that can be utilized to train the model where utilizing too many features may retrieve the overfitting and underfitting problem and are insufficient for the huge range of applications. For training, the large amount of data can give rise to random errors in the dataset. Instead of computing the relationship among the variables using DL and ML IDS, the dataset is converted into CSV format and comprises benign and malicious traffic in different domains.

VI. COMPARATIVE ANALYSIS AND EVALUATION

This section explains how the holdout strategy, which is used in machine learning to assess models, was used to validate the proposed approach. Throughout this method, the dataset is divided into two subsets, one of which is employed to train the model and another to assess it. For this experiment, the dataset is divided into three domains, with 30% used as testing and 70% as training. Table III illustrates the assigning of training and testing sets.

For the model's hyperparameters, training chooses a starting range of approximate values that are then tuned through trial and error to provide the optimal outcomes. The tuning method generates the desired results using 50 tuning epochs, a learning rate of 0.005, an output size of 70 for the LSTM and a drop-connect ratio of 0.01. Additionally, the layer's convolution filters were configured to have 32 in total. The maximum pooling length was limited to two, while the kernel size was tuned to three.

The Fig. 3(a) and 3(b) shows the comparative analysis of the baseline studies with proposed model and it has been observed that proposed model achieves 99.8% and 78.7% respectively; whereas 99.8%, 77.6%, 99.6%, 56.4%, 94.5%, 98.5% is computed for Decision Tree, K-Nearest Neighbor, Random Forest, Naive Bayes, CNN, LSTM.

The Fig. 4(a) and 4(b) indicates the comparative analysis of the baseline studies with proposed model which shows the improvement of 84% and 78.7% over the baseline studies that displays 65%, 81.7%, 68%, 68.4%, 69%, 68.7% computed for Decision Tree, K-Nearest Neighbor, Random Forest, Naive Bayes, CNN, LSTM.

The confusion metric of the proposed model is shown in Fig. 5.

VII. COMPUTATIONAL COMPLEXITY

Complexity means the number of trainable parameters (weights and bias) which says the higher the number of trainable parameters, the more the complexity of the model. We compute the time and space complexity of the proposed hybrid model on the SDN network in the CNN and LSTM

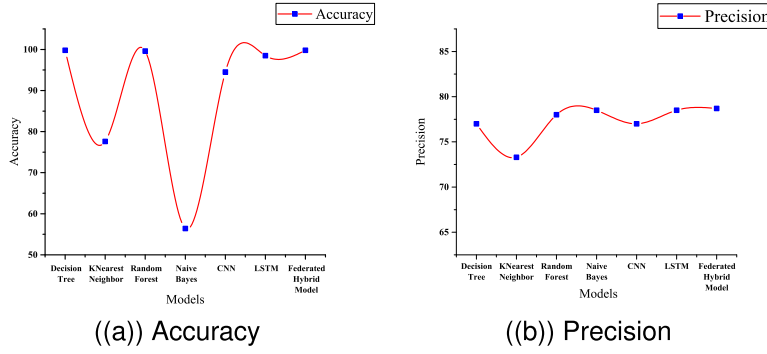


Fig. 3. Accuracy and Precision with regard to IDS.

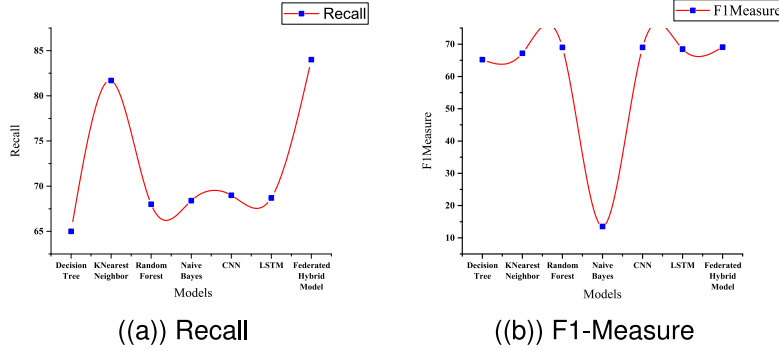


Fig. 4. Recall and F1-Measure with regard to IDS.

Hybrid CNN-LSTM model
Accuracy=0.998343
Precision=0.693653
Recall=0.767691
F1_Score=0.691642

	Normal	Exploits	Reconnaissance	DoS	Generic	Shellcode	Fuzzers	Worms	Backdoor	Analysis
Normal	967392 (1.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)
Exploits	0 (0.00)	22140 (1.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)
Reconnaissance	0 (0.00)	0 (0.00)	1377 (1.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)
DoS	0 (0.00)	0 (0.00)	0 (0.00)	3070 (1.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)
Generic	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	54544 (0.99)	892 (0.01)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)
Shellcode	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	5 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)
Fuzzers	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	14612 (0.91)	85 (0.01)	932 (0.08)	81 (0.01)
Worms	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)
Backdoor	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)
Analysis	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)	0 (0.00)

Fig. 5. Confusion metric of proposed model.

layers. The complexity of both the layers for one epoch is evaluated as $O(\sum_{i=1}^s m_{i-1} * p_i^2 * m_i * n_i^2)$; where s represents the number of convolutional layers; m_i shows the number of filters in the i^{th} layer; m_{i-1} denotes the number of input channels of the i^{th} layer; p_i is assigned the spatial size of filter and lastly n_i shows the spatial size of the output feature map. The time complexity for training the model is evaluated by inserting the computations on each training round: S signifies the number of devices that perform the computations on each training round; T refers to the majority of training rounds in which each device maintains the local dataset, and U denotes the size of the local minibatch used for the device updating. The computational complexity of baseline studies over the proposed model is shown in Table V.

Since the number of operations per input increases quadratically with kernel length and number, the CNN layer's complexity results. The length of the input, on the other hand, doesn't really impact the amount of storage needed by the network because LSTM is localised in space and time, and for each stage, the time complexity per weight is $O(1)$. Consequently, $O(X)$, where X is the number of weights in the LSTM, represents the overall complexity of LSTM every time stage.

The complexity of proposed hybrid model per time stage can be computed as the total of the complexity of CNN and LSTM layers: $O(\sum_{i=1}^s m_{i-1} * p_i^2 * m_i * n_i^2 + x)$ and for all the processes of training, the complexity is $O(\sum_{i=1}^s m_{i-1} * p_i^2 * m_i * n_i^2 + x) * l * c)$, where l depicts the input length and c depicts the number of epochs. Therefore, we can say that our proposed model has $O(\sum_{i=1}^s m_{i-1} * p_i^2 * m_i * n_i^2 + x) * l * c)$ complexity in the typical asymptotic notation.

VIII. CONCLUSION

In this article, the FRHIDS model is developed to detect the attacks on SDN network traffic generated by IoT domains against the data collected at the SDN network for recommender systems. Firstly, the novel algorithm is proposed for the HIDS model to train the SDN networks and federated cloud servers to protect data using a deep learning detection model. Furthermore, it enables HIDS to detect various attacks effectively. IDS and federated learning framework are applied to preserve the security and privacy of the model during training. The experiment is conducted and evaluated in terms of effectiveness, accuracy, precision, recall and f1-measure

TABLE V
OVERALL COMPLEXITY COMPARISON OF BASELINE STUDIES WITH PROPOSED FEDERATED LEARNING RECOMMENDER HYBRID MODEL

Reference	Model	Complexity
[8]	Decision Tree	$O(n \log n \log d)$
[7]	Naive Bayes	$O(n \log d \log c)$
[18]	KNearest Neighbor	$O(n \log d \log(n))$
[19]	Random Forest	$O(k \log n \log n \log d)$
[20]	LSTM	$X = 4IH + 4H^2 + 3H + HK$
[21]	CNN	$O(\sum_{i=1}^s m_{i-1} * p_i^2 * m_i * n_i^2)$
Our Model	Federated RHIDS	$O(\sum_{i=1}^s m_{i-1} * p_i^2 * m_i * n_i^2 + x) * l * c)$

of the proposed model as compared to the other baseline solutions. This study has shown 12% improvement. In the future,

- 1) Machine and Deep learning can be proposed to the next generation wireless networks, i.e., 6G and preservation of privacy will give pivotal importance in 6G which therefore describes the homomorphic encryption against the malicious global aggregation server.
- 2) The further improvement and exploration will be done on the methods to detect the attacks in federated recommendation learning.

REFERENCES

- [1] M. Ammad-Ud-Din et al., "Federated collaborative filtering for privacy-preserving personalized recommendation system," 2019, *arXiv:1901.09888*.
- [2] M. Fang, G. Yang, N. Z. Gong, and J. Liu, "Poisoning attacks to graph-based recommender systems," in *Proc. 34th Annu. Comput. Security Appl. Conf.*, 2018, pp. 381–392.
- [3] D. Rong, S. Ye, R. Zhao, H. N. Yuen, J. Chen, and Q. He, "FedRecAttack: Model poisoning attack to federated recommendation," in *Proc. IEEE 38th Int. Conf. Data Eng. (ICDE)*, 2022, pp. 2643–2655.
- [4] S. Nanda, F. Zafari, C. DeCusatis, E. Wedaa, and B. Yang, "Predicting network attack patterns in SDN using machine learning approach," in *Proc. IEEE Conf. Netw. Funct. Virtualizat. Softw. Defined Netw. (NFV-SDN)*, 2016, pp. 167–172.
- [5] A. Thakkar and R. Lohiya, "A review on machine learning and deep learning perspectives of IDS for IoT: Recent updates, security issues, and challenges," *Arch. Comput. Methods Eng.*, vol. 28, no. 4, pp. 3211–3243, 2021.
- [6] P. Dhiman et al., "A novel deep learning model for detection of severity level of the disease in citrus fruits," *Electronics*, vol. 11, no. 3, p. 495, 2022.
- [7] S. Meftah, T. Rachidi, and N. Assem, "Network based intrusion detection using the UNSW-NB15 dataset," *Int. J. Comput. Digit. Syst.*, vol. 8, no. 5, pp. 478–487, 2019.
- [8] S. Bagui, E. Kalaimannan, S. Bagui, D. Nandi, and A. Pinto, "Using machine learning techniques to identify rare cyber-attacks on the UNSW-NB15 dataset," *Security Privacy*, vol. 2, no. 6, 2019, Art. no. e91.
- [9] D. Kumar, V. Kukreja, V. Kadyan, and M. Mittal, "Detection of DoS attacks using machine learning techniques," *Int. J. Veh. Auton. Syst.*, vol. 15, nos. 3–4, pp. 256–270, 2020.
- [10] D. K. Talapula, K. K. Ravulakollu, M. Kumar, and A. Kumar, "SAR-BSO meta-heuristic hybridization for feature selection and classification using DBNover stream data," *Artif. Intell. Rev.*, vol. 56, pp. 14327–14365, Dec. 2023.
- [11] Y. Fan, Y. Li, M. Zhan, H. Cui, and Y. Zhang, "IoTDefender: A federated transfer learning intrusion detection framework for 5G IoT," in *Proc. IEEE 14th Int. Conf. Big Data Sci. Eng. (BigDataSE)*, 2020, pp. 88–95.
- [12] B. Li, Y. Wu, J. Song, R. Lu, T. Li, and L. Zhao, "DeepFed: Federated deep learning for intrusion detection in industrial cyber-physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5615–5624, Aug. 2021.
- [13] S. I. Popoola, R. Ande, B. Adebisi, G. Gui, M. Hammoudeh, and O. Jogunola, "Federated deep learning for zero-day botnet attack detection in IoT-edge devices," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3930–3944, Mar. 2022.
- [14] M. M. Rashid, S. U. Khan, F. Eusufzai, M. A. Redwan, S. R. Sabuj, and M. Elsharief, "A federated learning-based approach for improving intrusion detection in Industrial Internet of Things networks," *Network*, vol. 3, no. 1, pp. 158–179, 2023.
- [15] C. Choudhary, I. Singh, and M. Kumar, "SARWAS: Deep ensemble learning techniques for sentiment based recommendation system," *Exp. Syst. Appl.*, vol. 216, Apr. 2023, Art. no. 119420.
- [16] A. Meliboev, J. Alikhanov, and W. Kim, "Performance evaluation of deep learning based network intrusion detection system across multiple balanced and imbalanced datasets," *Electronics*, vol. 11, no. 4, p. 515, 2022.
- [17] S. Potluri, S. Ahmed, and C. Diedrich, "Convolutional neural networks for multi-class intrusion detection system," in *Proc. Int. Conf. Mining Intell. Knowl. Explor.*, 2018, pp. 225–238.
- [18] S. M. Kasongo and Y. Sun, "Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset," *J. Big Data*, vol. 7, no. 1, pp. 1–20, 2020.
- [19] R. A. Disha and S. Waheed, "Performance analysis of machine learning models for intrusion detection system using Gini impurity-based weighted random forest (GIWRF) feature selection technique," *Cybersecurity*, vol. 5, no. 1, pp. 1–22, 2022.
- [20] S. Haider et al., "A deep CNN ensemble framework for efficient DDos attack detection in software defined networks," *IEEE Access*, vol. 8, pp. 53972–53983, 2020.
- [21] M. S. ElSayed, N.-A. Le-Khac, M. A. Albahar, and A. Jurcut, "A novel hybrid model for intrusion detection systems in SDNs based on CNN and a new regularization technique," *J. Netw. Comput. Appl.*, vol. 191, Oct. 2021, Art. no. 103160.

Himanshi Babbar received the M.C.A. and Ph.D. degrees from Chitkara University (Punjab Campus), India, in 2015 and 2021, respectively, and the Doctoral degree in computer applications from Zayed University, UAE, in 2022. She is working as an Assistant Professor of Research in CSE with Chitkara University (Punjab Campus).

Shalli Rani (Senior Member, IEEE) received the Ph.D. degree from Punjab Technical University, Jalandhar, in 2017, and the M.Tech. degree in computer science from JRNv, Udaipur, India, in 2007. She was a Postdoctoral Fellow with Manchester Metropolitan University, U.K., in 2023. She is a Professor of CSE with Chitkara University (Punjab Campus), India. She has more than 18 years of teaching experience. Her main areas of interest are WSN, IoT, machine and deep learning, and security.